

Intelligent Robotics

Coppeliasim project

Project Abstract

Robotic arms are very widely used in many industries, and while they may seem instinctual in the way they operate similar to our hands, they are way more complex than they seem. In order for a robot to know its target, it needs to calculate its cumulative trajectory starting from its base all the way up to the tip which is meant to coincide with the target coordinate. In order for this to be possible, most robotic arms rely on forward and inverse kinematics to efficiently perform the calculations required to derive these values.

This project provides a walkthrough of how a generic robotic arm doesn't simply look at the spatial data of its target position, but rather how it has to calculate how much each joint should move, and in which degree, to efficiently reach the target coordinate.

The robotic arm uses these methods to iteratively test all the infinitesimal possibilities of joint combinations, and chooses the form with the least error.

Denavit-Hartenberg parameters

Each joint can be characterized by four main metrics

θ - joint angle

α - link twist

a - link length

d - link offset

In setting up my robotic arm, some of these metrics are proprietary to each arm, such as the arm-length, while other depend on my personal initialization

```
# Forward Kinematics Function:

def FK(theta, alpha, a, d):

    #conversion of theta to radians
    theta = theta * np.pi/180

    #conversion of alpha to radians
    alpha = alpha * np.pi/180

    Cth = np.cos(theta) #Cos_theta
    Sth = np.sin(theta) #Sin_theta
    Calp = np.cos(alpha) #Cos_alpha
    Salp = np.sin(alpha) #Sin_alpha

    #Forward Transformation Matrix from the DH-Table
    T = np.array([
        [Cth, -Sth*Calp, Sth*Salp, a*Cth],
        [Sth, Cth*Calp, -Cth*Salp, a*Sth],
        [0, Salp, Calp, d],
        [0, 0, 0, 1]
    ])

    return np.round(T, 3)
```

In this Excerpt of the Code, I am creating a function to parameterize the DH-table, which actively serves the forward kinematics.

```

T0_0 = FK(0, 0, 0, 0)
T0_1 = FK(q[0], -90, 0, 0.34 )
T1_2 = FK(q[1], 90, 0, 0 )
T2_3 = FK(q[2], 90, 0, 0.4 )
T3_4 = FK(q[3], -90, 0, 0 )
T4_5 = FK(q[4], -90, 0, 0.4 )
T5_6 = FK(q[5], 90, 0, 0 )
T6_7 = FK(q[6], 0, 0, 0.126)

# Get Transformation from end-effector frame to base frame

T0_2 = T0_1 @ T1_2
T0_3 = T0_1 @ T1_2 @ T2_3
T0_4 = T0_1 @ T1_2 @ T2_3 @ T3_4
T0_5 = T0_1 @ T1_2 @ T2_3 @ T3_4 @ T4_5
T0_6 = T0_1 @ T1_2 @ T2_3 @ T3_4 @ T4_5 @ T5_6

T0_7 = T0_1 @ T1_2 @ T2_3 @ T3_4 @ T4_5 @ T5_6 @ T6_7

print("Analytical (T0_7) = \n", np.round(T0_7, 2))

```

In this part of the code, i am assembling the robotic arms in terms of its DH parameters to be able to enforce FK and IK

I personally chose to predefine all the angles of the robotic arm to zero because this decreases the error rate of the iterations to find the lowest error.

By doing this, I'm preventing my code from running semi-infinitely, just in order to accustom for the change in code angles.

Actual Transformation of the End Effector

```
[[ 1.  -0.  0.  0.01]]
```

```
[ 0.  1. -0.  0. ]
```

```
[-0.  0.  1.  1.21]
```

```
[ 0.  0.  0.  1. ]]
```

This is the output of my run of the above function, clarifying the coordinates of the end-point of the robotic arm, aka the final joint/end-effector.

These coordinates are meant to be eventually compared to the analytical calculations, however a small offset is inevitable due to the physics in Coppelism that changes the coordinates.

```
joint_names = [f'joint_{i+1}' for i in range(7)]

print ("Joint Names are: ", joint_names)
```

I am setting the joint names in order to properly address them in the code one running the iterations.

```
# Getting the joints Handels:

joint_handles=[sim.getObjectHandle(name) for name in joint_names]

# Setting both of the End Effector & Base Handles:

Base = sim.getObjectHandle("joint_1")

EE = sim.getObjectHandle("joint_7")

EE_Dummy = sim.getObjectHandle("EE_Dummy")

print("Joint Handles are: ", joint_handles)
```

This part of the code connects the names we derived earlier to the true joint objects in Coppeliasim, in order to explicitly address the base and end-effector (and the other joints).

The EE_Dummy is the dummy object I created and attached to the tip of the robot, to verify the accuracy of how my robot performed.

```
for i in range(len(joint_handles)):  
  
    sim.setObjectInt32Parameter(joint_handles[i], 2001, ~0)
```

This allows the robotic arms to listen to any changes.

```
sim.startSimulation()  
  
#Analytical:  
  
print("T0_7")  
  
print("Analytical Transformation of the End Effector\n", np.round(T0_7,  
2))  
  
print("===== \n")  
  
#Actual  
  
for i in range(7):  
  
    sim.setJointTargetPosition(joint_handles[i], q[i]*np.pi/180)  
  
time.sleep(0.5)
```

```

T_EE = sim.getObjectMatrix(EЕ_Dummy,-1)

# Convert the flattened 3x4 matrix into a 4x4 matrix

T_EE = np.array(T_EE).reshape(3, 4)      # Reshape as 3x4

T_EE = np.vstack((T_EE, [0, 0, 0, 1]))    # Add the last row [0, 0, 0, 1]

print("Actual Transformation of the End Effector\n", np.round(T_EE, 2))

```

This part is meant to assess how accurate my math is, and outputs the difference between the actual coordinate of the end effector, and the analytical derivation through the FK calculations

T0_7

Analytical Transformation of the End Effector

```

[[1.  0.  0.  0. ]
 [0.  1.  0.  0. ]
 [0.  0.  1.  1.27]
 [0.  0.  0.  1.  ]]

```

=====

Actual Transformation of the End Effector

```

[[ 1.  -0.  0.  0.01]
 [ 0.   1. -0.  0. ]
 [-0.   0.  1.  1.21]
 [ 0.   0.  0.  1.  ]]

```

The above output shows the difference, which in my case has very minor differences, which exist due to the physics in Coppeliasim.

```
def Jacobian(T0, T1, T2, T3, T4, T5, T6, T7):

    linksToZero = [T0, T1, T2, T3, T4, T5, T6, T7]

    Jp=np.array([], [], [])

    Jr=np.array([], [], [])

    Pos_T07=np.array([T7[0][3], T7[1][3], T7[2][3]])

    for i in range(7):

        R = np.array([linksToZero[i][0][2], linksToZero[i][1][2],
linksToZero[i][2][2]])

        P = np.array([linksToZero[i][0][3], linksToZero[i][1][3],
linksToZero[i][2][3]])

        Psubtract=np.subtract(Pos_T07, P)

        Jpp=np.cross(R, Psubtract)

        Jp=np.hstack((Jp, np.atleast_2d(Jpp).T))

        Jr=np.hstack((Jr, np.atleast_2d(R).T))
```

```
J=np.concatenate((Jp, Jr))

return J
```

This code initializes variables that will be storing the output for each iteration.

The for loop calculates the rotation and lengths of each joint iteratively, and it also subtracts at each loop from the previous joint to be able to calculate the end effector length. It crosses the values together and stacks them to create a 6x7 matrix that displays the three possible angles of rotation with the three possible dimension lengths by the seven possible joints in my robotic arm.

```
def deltaX(T7, RD, PD):

    # Create a 3x3 identity matrix

    I = np.eye(3)

    # Extract the 3x3 rotation matrix from the transformation matrix (T7)

    R0 = np.array([T7[0][0:3], T7[1][0:3], T7[2][0:3]])

    # Extract the position matrix from the transformation matrix (T7)

    P0 = np.array([T7[0][3], T7[1][3], T7[2][3]])

    # Compute the transpose of the current rotation matrix (R0)

    R0T = R0.transpose()

    # Compute the rotation difference matrix: RD * R0T - I
```

```

# RD is the desired rotation matrix, R0 is the current rotation matrix

skew_w = RD @ ROT - I

# Extract the rotation error from the skew matrix

vec_w = np.array([[skew_w[2][1]], [skew_w[0][2]], [skew_w[1][0]]])

# Compute the difference between desired (PD) and current positions
(P0)

deltax = np.array([PD[0] - P0[0], PD[1] - P0[1], PD[2] - P0[2]])

# Concatenate the positional difference (deltax) and angular
difference (vec_w)

# The result is a 6x1 array containing both linear and angular errors

deltax = np.concatenate((deltax, vec_w))

```

This part of the code is responsible for calculating the error after the previous code has generated the values. It looks at these six values and compares them to the target value. And it does simple subtraction for the x, y, z degrees, while for the rotation, it has to perform a certain calculation to be able to calculate the angular differences, and it outputs this error on a 6x1 matrix.

```

# Inverse Kinematics Function

def IK(J, deltax, Q0):

    # Compute the pseudo-inverse of the Jacobian matrix (J)

    # This is used to map the 6x1 end-effector error vector (deltax) to
joint space

    Jplus = np.round(np.linalg.pinv(J), 2) # Round to 2 decimal places for
stability

    # Compute the joint angle increments (delta Q)

    # This is the product of the pseudo-inverse Jacobian and the error
vector

    JJ = Jplus @ deltax

    # Update the current joint angles (Q0) by adding the increments

    Qd = Q0 + JJ # # Update joint angles

    # Return the updated joint angles

    return Qd

```

This code is responsible for splitting these six margin errors of the degrees onto the seven joints of the robot, deciding which joint needs to move with what extent in order to minimize this error and make up for the offset.

```

# Example for a desired Position:
PosD = np.array([[ -0.872], [+0.42452], [+0.00515]])

# Use a valid desired rotation matrix for the end effector.
# The previous matrix was invalid and could make the arm rotate
incorrectly.
RotD = np.eye(3)

# initial joint angles
Q0 = np.array([[90], [0], [-90], [0], [90], [-180], [0]], dtype=float)

# Helper to compute all cumulative base-frame transforms for the current
joint vector

def compute_T0_frames(q):
    T0_1 = FK(q[0, 0], -90, 0, 0.34)
    T1_2 = FK(q[1, 0], 90, 0, 0)
    T2_3 = FK(q[2, 0], 90, 0, 0.4)
    T3_4 = FK(q[3, 0], -90, 0, 0)
    T4_5 = FK(q[4, 0], -90, 0, 0.4)
    T5_6 = FK(q[5, 0], 90, 0, 0)
    T6_7 = FK(q[6, 0], 0, 0, 0.126)

    T0_2 = T0_1 @ T1_2
    T0_3 = T0_2 @ T2_3
    T0_4 = T0_3 @ T3_4
    T0_5 = T0_4 @ T4_5
    T0_6 = T0_5 @ T5_6
    T0_7 = T0_6 @ T6_7

    return T0_1, T0_2, T0_3, T0_4, T0_5, T0_6, T0_7

T0_1, T0_2, T0_3, T0_4, T0_5, T0_6, T0_7 = compute_T0_frames(Q0)
Jacob = Jacobian(T0_0, T0_1, T0_2, T0_3, T0_4, T0_5, T0_6, T0_7)
delX = deltaX(T0_7, RotD, PosD)

q = Q0.copy()
normdelX = np.linalg.norm(delX)
iteration = 0
max_iter = 100

```

```

while normdelX > 0.01 and iteration < max_iter:
    q = IK(Jacob, delX, q)
    T0_1, T0_2, T0_3, T0_4, T0_5, T0_6, T0_7 = compute_T0_frames(q)
    Jacob = Jacobian(T0_0, T0_1, T0_2, T0_3, T0_4, T0_5, T0_6, T0_7)
    delX = deltaX(T0_7, RotD, PosD)
    normdelX = np.linalg.norm(delX)
    iteration += 1

print(f"IK finished after {iteration} iterations,
normdelX={normdelX:.4f}")
print("Final joint angles (degrees):", q.T)

```

This is the main loop that repeats all the previously discussed functions iteratively.

```

# Calculate the Jacobian matrix for the current joint configuration
# T0_0 to T0_7 are the transformation matrices from the base frame to each
joint or end-effector frame
Jacob = Jacobian(T0_0, T0_1, T0_2, T0_3, T0_4, T0_5, T0_6, T0_7)

# Calculate the error vector (deltaX) between the desired and current
end-effector pose
# T0_7 is the current end-effector transformation matrix
# RotD is the desired rotation matrix
# PosD is the desired position
delX = deltaX(T0_7, RotD, PosD)

# Calculate the norm (magnitude) of the error vector (delX)
# This measures how far the end-effector is from the desired pose
normdelX = np.linalg.norm(delX)
print('Norm Delta X\n', normdelX)

# Round the computed joint angles (q) to two decimal places for numerical
stability
q = np.round(q, 2)
print('Q desired:\n', q.item((0, 0)))

# Loop through all 7 joints to set the target positions in the simulator

```

```
# Converts the joint angles from degrees to radians and sets them in
CoppeliaSim
for i in range(7):
    sim.setJointTargetPosition(joint_handles[i], float(q[i, 0]) * np.pi /
180)
```

The calculated error values are simplified here in this part of the code and are converted into the suitable form, such as switching between radian and angle, and simplifying these six error values into one main value, vector-based, and sends it to CopeliaSim to repeat the process and minimize the error difference.